



STP 协议设计文档

基于 UART 的可靠传输协议

版本	1.1.0
作者	固件开发组
部门	芯片安全事业部
日期	2025 - 09 - 03
分类	内部文档

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2025 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2025 深圳市纽创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

修订历史

版本	修改日期	描述
1.0.0	2025-08-22	初版
1.1.0	2025-09-03	• 增加服务端和客户端角色，区分默认状态 • 增加字符串帧功能

目录

1 简介	7
2 协议设计	7
2.1 角色定义	7
2.1.1 客户端 (Client)	7
2.1.2 服务端 (Server)	7
2.2 帧结构	7
2.3 帧类型	8
2.3.1 标准数据帧类型	8
2.3.2 字符串输出帧	8
2.4 状态机	9
2.4.1 客户端状态机	9
2.4.2 服务端状态机	10
2.4.3 状态转换说明	11
3 实现架构	11
3.1 C 语言实现架构	11
3.1.1 分层结构	11
3.1.2 核心数据结构	12
3.1.3 C 语言协议接口	12
3.1.4 硬件抽象层接口	13
3.2 Python 实现架构	13
3.2.1 类设计结构	13
3.2.2 异常处理机制	14
3.2.3 内部状态管理	14
4 协议实现规范	14
4.1 CRC 校验算法	14
4.1.1 算法参数	14
4.1.2 实现示例	14
4.2 超时机制	15
4.2.1 超时参数配置	15
4.2.2 超时处理策略	16
4.3 错误处理机制	16
4.3.1 错误类型定义	16
4.3.2 错误处理流程	16
4.4 缓存机制与数据管理	16
4.4.1 接收数据缓存	16
4.4.2 数据流处理策略	16
4.4.3 实现示例	17
4.4.4 模式切换管理	17
5 通信流程	17
5.1 初始化与同步流程	17
5.1.1 初始化流程	18

5.1.2 重新同步机制	18
5.1.3 首次通信优化	18
5.2 数据传输流程	19
5.2.1 发送端流程	19
5.2.2 接收端流程	19
6 总结	19

表目录

表 1	帧结构定义	7
表 2		8
表 3	字符串帧结构定义	8
表 4	客户端与服务端状态机对比	11
表 5	C 语言实现分层架构	11
表 6	Python 异常类型定义	14
表 7	CRC16 算法参数	14
表 8	超时参数配置	15
表 9	错误码定义	16
表 10	接收缓存参数	16
表 11	初始化流程	18

1 简介

基于 UART 的可靠传输协议

- 半双工，有发送/接收两种状态
- 基于帧结构
- 支持校验与重发

2 协议设计

2.1 角色定义

协议区分服务端和客户端两种角色：

2.1.1 客户端 (Client)

- **初始模式**：发送模式 (Send Mode)
- **初始化行为**：启动时主动发送 RESYNC_REQ 进行握手
- **RESYNC 权限**：只有客户端可以发送 RESYNC_REQ 请求重新同步
- **典型场景**：主动发起数据传输的设备

2.1.2 服务端 (Server)

- **初始模式**：接收模式 (Receive Mode)
- **初始化行为**：等待客户端的 RESYNC_REQ，然后回复 RESYNC_ACK
- **RESYNC 权限**：只能被动响应 RESYNC_REQ，发送 RESYNC_ACK
- **重置行为**：收到 RESYNC_REQ 后，重置到初始的接收模式
- **典型场景**：被动响应的设备，如传感器、回显服务器等

2.2 帧结构

每个帧作为一个整体进行交互，负载数据的传输基于不同类型的帧交互来完成。

表 1 帧结构定义

字段	长度 (字节)	说明
起始字节	1	0x5A, 用于判断帧起始条件
类型	1	指示帧的类型
负载长度	1	最多支持 255 字节负载数据
负载数据	N	只有数据帧支持负载数据
校验	2	前面所有字段的 CRC16 校验值

2.3 帧类型

2.3.1 标准数据帧类型

帧类型由帧的类型字段来定义，具体定义如下：

帧类型字段位	类型	说明
bit0/1	错误码	用于错误帧中指示具体错误
bit2	同步响应 RSYNC_ACK	服务端收到 RSYNC_REQ 后发送确认，并重置到接收模式
bit3	同步请求 RSYNC_REQ	仅客户端可发送 的重新同步请求
bit4	数据帧 DATA	附带发送的数据
bit5	确认帧 ACK	确认收到数据帧
bit6	切换方向 SWITCH	发送方从发送状态切换为接收状态
bit7	错误帧 ERR	收到的帧错误，低 2 位包含错误码

2.3.2 字符串输出帧

除了标准的数据协议帧外，协议还支持独立的字符串输出帧，用于直接输出字符串信息，不参与协议的状态机交互。

2.3.2.1 字符串帧结构

字符串帧采用与标准数据帧完全不同的格式：

表 3 字符串帧结构定义

字段	长度（字节）	说明
起始标识	2	固定为 0xFE 0xF0，用于区别标准数据帧
字符串内容	变长	待输出的字符串内容，无长度限制
结束标识	2	固定为 0xFE 0xFE，标识字符串结束

2.3.2.2 字符串帧特点

1. **独立性**：字符串帧不影响标准数据协议的状态机运行
2. **简化性**：无 CRC 校验、无长度字段、无重传机制
3. **实时性**：收到即输出，适合调试信息和状态提示
4. **单向性**：仅用于输出，不需要确认响应

2.3.2.3 字符串帧处理逻辑

接收端在帧起始字节检测时同时检测两种帧类型：

1. 检测到 0x5A：按标准数据帧处理
2. 检测到 0xFE：
 - 继续检测第二字节是否为 0xF0
 - 若是，则进入字符串帧处理流程
 - 若不是，则忽略并继续检测

字符串帧处理流程：

1. 验证起始字节 0xFE 0xF0

2. 读取后续字节到缓冲区
3. 检测结束字节 `0xFE 0xFE`
4. 调用字符串输出回调函数
5. 处理缓冲区满和超时情况

2.3.2.4 字符串帧示例

发送字符串 `Hello World` 的字符串帧：

```
1| 0xFE 0xF0 'H' 'e' 'l' 'l' 'o' ' ' 'W' 'o' 'r' 'l' 'd' 0xFE 0xFE
```

对应的十六进制：

```
1| FE F0 48 65 6C 6C 6F 20 57 6F 72 6C 64 FE FE
```

2.4 状态机

协议使用状态机来管理半双工通信。由于客户端和服务端的角色不同，它们的状态机实现也有所区别：

2.4.1 客户端状态机

客户端作为主动方，具有发起 RESYNC 的能力：

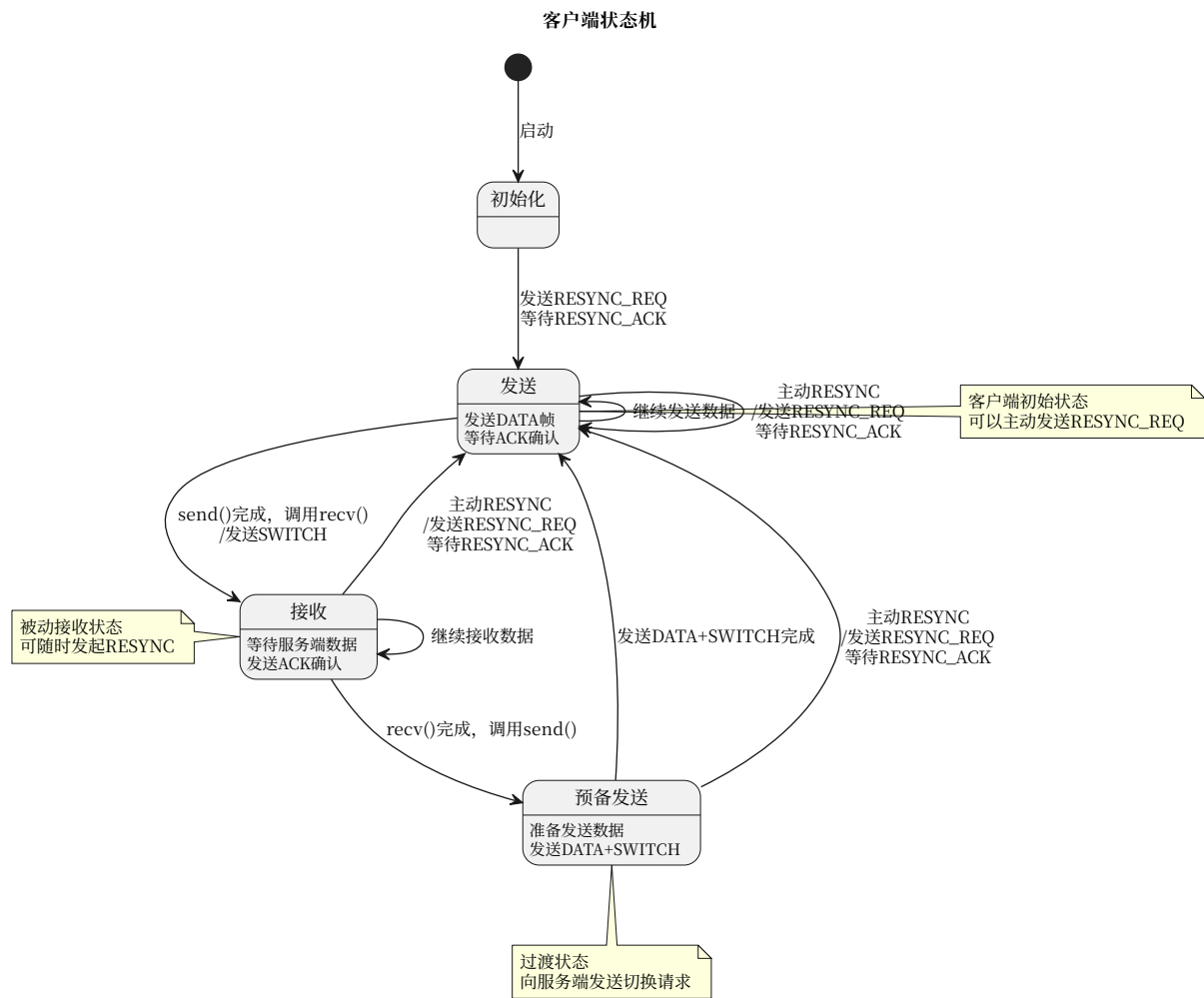


图 1 客户端状态机

2.4.2 服务端状态机

服务端作为被动方，只能响应客户端的请求：

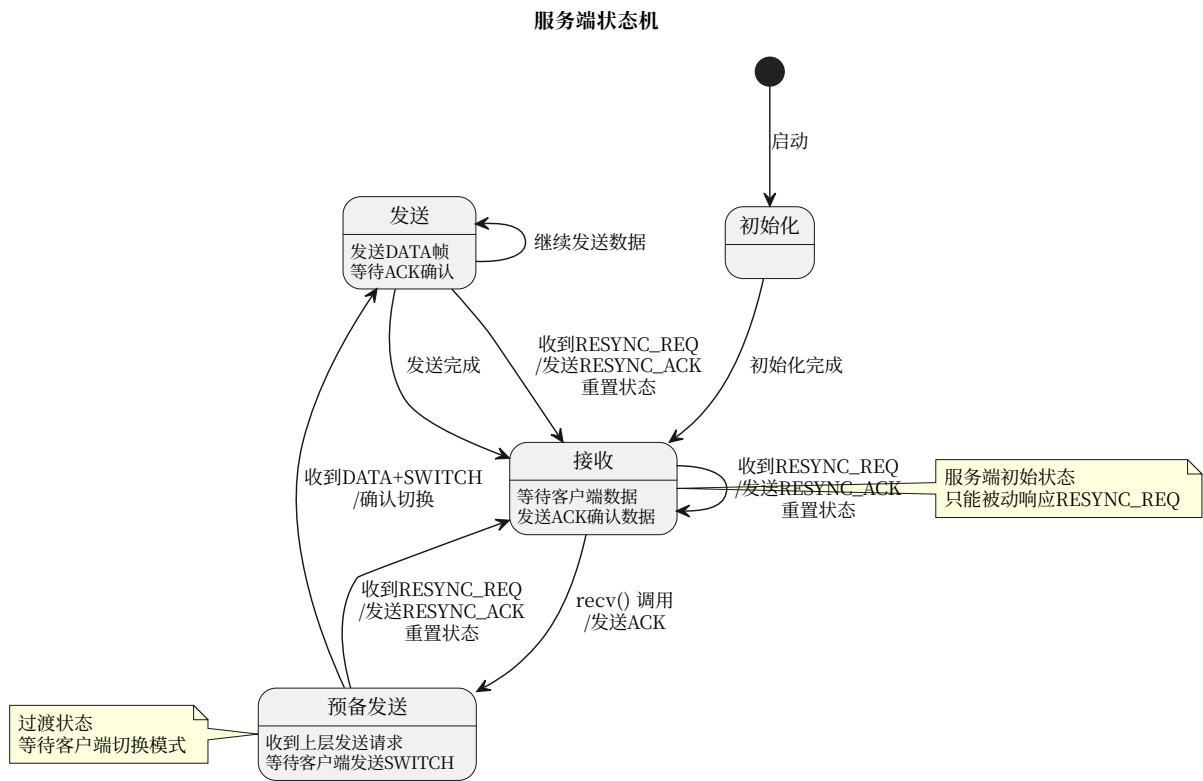


图 2 服务端状态机

2.4.3 状态转换说明

表 4 客户端与服务端状态机对比

角色	初始状态	主要转换	RESYNC 行为
客户端	发送模式	发送→接收→发送	主动发送 RESYNC_REQ
服务端	接收模式	接收→预备发送→发送→接收	被动响应 RESYNC_ACK 并重置

3 实现架构

3.1 C 语言实现架构

C 语言实现采用分层架构设计，确保协议层与硬件抽象层的解耦：

3.1.1 分层结构

表 5 C 语言实现分层架构

层次	文件	功能
应用层	用户代码	调用 STP 协议 API 进行数据传输
协议层	strans_pro.h/uart_pro.c	实现 STP 协议核心逻辑、状态机管理
硬件抽象层	uart_hal.h	定义 UART 硬件接口规范
平台适配层	win_uart_hal.c 等	具体平台的 UART 实现

3.1.2 核心数据结构

协议层使用 `uart_pro_st` 结构体管理协议状态:

```
1| typedef struct {
2|     uint8_t recv_buf[MAX_PAYLOAD_SIZE]; // 接收缓存, 255字节
3|     uint8_t recv_data_size;             // 缓存中的数据长度
4|     uint8_t recv_data_off;              // 缓存读取偏移
5|     bool_t send_mode;                   // 当前模式: TRUE=发送模式, FALSE=接收模式
6|     bool_t first_init;                  // 首次初始化标志
7| } uart_pro_st;
```

3.1.3 C 语言协议接口

协议通过 `strans_pro.h` 头文件提供标准化的 C 语言 API:

```
1| // 字符串输出回调函数类型定义
2| typedef void (*stp_string_output_cb)(const char *str, uint32_t len);
3|
4| // 协议初始化与清理
5| uint32_t stp_init(bool_t is_client, stp_string_output_cb string_cb); // 初始化协议,
指定角色和字符串输出回调
6| void stp_deinit(void); // 关闭协议
7|
8| // 数据传输接口
9| uint32_t stp_send(const uint8_t *data, uint32_t size); // 发送数据
10| uint32_t stp_rcv(uint8_t *buf, uint32_t size); // 接收数据
11| uint32_t stp_flush(void); // 刷新发送缓存
12|
13| // 字符串帧接口
14| uint32_t stp_send_string(const uint8_t *data, uint32_t size); // 发送字符串帧
15|
16| // 协议控制接口
17| void stp_reset_comm(void); // 手动重置通信状态
18|
19| // 系统接口 (需用户实现)
20| uint32_t stp_get_time_ms(void); // 获取系统时间戳
```

参数说明:

- `is_client`: TRUE 表示客户端角色, FALSE 表示服务端角色
- `string_cb`: 字符串输出回调函数, 收到字符串帧时调用。如为 NULL 则忽略字符串帧
- 客户端: 初始为发送模式, 主动发起 RESYNC 握手
- 服务端: 初始为接收模式, 被动响应 RESYNC 握手

字符串输出回调函数:

```
1| void string_output_handler(const char *str, uint32_t len) {
2|     printf("Received string: %.*s\n", len, str);
3| }
```

3.1.4 硬件抽象层接口

HAL 层定义了最小化的 UART 接口，便于跨平台移植：

```
1| uint32_t uart_init(void);           // 初始化UART硬件
2| uint32_t uart_send_byte(uint8_t val); // 发送单字节
3| uint32_t uart_peek_byte(uint8_t* val); // 非阻塞读取单字节
4| void uart_deinit(void);             // 关闭UART硬件
```

3.2 Python 实现架构

Python 实现采用面向对象设计，提供简洁的 API 接口：

3.2.1 类设计结构

`STPUart` 类封装了完整的协议功能：

```
1| class STPUart:
2|     def __init__(self, is_client: bool, port: str, baudrate: int = 57600,
3|                   string_callback: callable = None)
4|     def send(self, data: bytes) -> None
5|     def recv(self, size: int) -> bytes
6|     def send_string(self, data: bytes) -> None # 发送字符串帧
7|     def close(self)
8|     def reset_comm(self) -> None             # 手动重置通信状态
9|     def resync(self) -> None                  # 客户端主动重新同步
```

参数说明：

- `string_callback`: 字符串输出回调函数，收到字符串帧时调用。如为 `None` 则忽略字符串帧
- 回调函数参数为 `bytes` 类型，由实现者决定如何解码

字符串输出回调函数示例：

```
1| def string_output_handler(data: bytes):
2|     try:
3|         text = data.decode('utf-8')
4|         print(f"Received string: {text}")
5|     except UnicodeDecodeError:
6|         print(f"Received binary: {data.hex()}")
7|
8| uart = STPUart(is_client=True, port="/dev/ttyUSB0",
string_callback=string_output_handler)
```

3.2.2 异常处理机制

Python 实现定义了专门的协议异常类型：

表 6 Python 异常类型定义

异常类型	说明
STPTimeoutError	超时错误
STPCrcError	CRC 校验错误
STPResync	重同步请求
STPResyncFailed	重同步失败
STPError	通用协议错误

3.2.3 内部状态管理

Python 实现使用实例变量管理协议状态：

```
1| self.__send_mode: bool          # 发送/接收模式
2| self.__recv_buf: bytearray       # 接收数据缓存
3| self.__recv_size: int            # 缓存数据大小
4| self.__recv_off: int             # 缓存读取偏移
5| self.__init: bool               # 初始化状态标志
```

4 协议实现规范

4.1 CRC 校验算法

协议采用 CRC16 校验算法确保数据完整性：

4.1.1 算法参数

表 7 CRC16 算法参数

参数	值
多项式	0x1021 (CRC-16-CCITT)
初始值	0x0000
字节序	大端序（高字节在前）
计算范围	帧头+负载数据（不包含 CRC 本身）

4.1.2 实现示例

C 语言实现：

```
1| static uint16_t up_calc_crc(const uint8_t *data, uint32_t size, uint16_t init)
2| {
3|     uint16_t val = init;
4|     uint16_t poly = 0x1021;
```

```

5|     uint16_t wchar = 0;
6|
7|     while (size--) {
8|         wchar = *(data++);
9|         val ^= (wchar << 8);
10|
11|         for (int i = 0; i < 8; i++) {
12|             if (val & 0x8000) {
13|                 val = (val << 1) ^ poly;
14|             } else {
15|                 val = val << 1;
16|             }
17|         }
18|     }
19|     return val;
20| }

```

Python 实现:

```

1| def calc_crc(data: bytes | bytearray) -> bytes:
2|     val = 0x0000
3|     poly = 0x1021
4|
5|     for c in data:
6|         val ^= (c << 8) & 0xFFFF
7|         for _ in range(8):
8|             if val & 0x8000:
9|                 val = ((val << 1) & 0xFFFF) ^ poly
10|            else:
11|                val = (val << 1) & 0xFFFF
12|     return val.to_bytes(2, "big")

```

4.2 超时机制

协议实现了分层的超时控制机制:

4.2.1 超时参数配置

表 8 超时参数配置

超时类型	C 实现	Python 实现	说明
字节超时	100ms	500ms	单字节接收的最大等待时间
块超时	10s	30s	等待帧开始的最大时间
重传次数	5 次	5 次	发生错误时的最大重传次数

4.2.2 超时处理策略

1. **字节超时**：在接收帧的过程中，如果等待下一个字节超过设定时间，则认为传输中断，发送超时错误帧。
2. **块超时**：等待新帧的起始字节时，如果超过设定时间，则可能触发协议重新同步。
3. **重传机制**：接收到错误帧或发生超时，自动重传当前帧，最多重传 5 次。

4.3 错误处理机制

4.3.1 错误类型定义

表 9 错误码定义

错误码	错误类型	处理策略
0x01	CRC 错误	重传当前帧
0x02	字符超时	重传当前帧
0x03	类型错误	重传当前帧

4.3.2 错误处理流程

1. **错误检测**：接收端检测到 CRC 错误、超时或无效帧类型时，发送对应的错误帧。
2. **错误响应**：发送端收到错误帧后，根据错误码重传相应的数据帧。
3. **错误计数**：连续发生错误时进行计数，超过最大重传次数（5 次）则终止通信。
4. **协议恢复**：严重错误时可触发重新同步流程，重新建立通信连接。

4.4 缓存机制与数据管理

协议实现了智能的数据缓存机制，支持任意大小的数据收发：

4.4.1 接收数据缓存

表 10 接收缓存参数

参数	大小	说明
缓存大小	255 字节	与最大帧负载大小一致
缓存管理	offset 指针	支持部分读取和数据拼接
缓存状态	size + offset	动态跟踪可用数据长度

4.4.2 数据流处理策略

1. **大数据分块传输**：
 - 发送端：自动将大于 255 字节的数据分块发送
 - 接收端：透明拼接多个数据帧，对用户呈现连续数据流
2. **部分读取支持**：
 - 用户可以请求任意长度的数据
 - 协议自动管理缓存中的剩余数据

- 支持多次小量读取和一次大量读取混合使用

3. 缓存优化:

- 避免不必要的拷贝
- 使用 offset 指针管理已读取的数据
- 自动清理已消费的缓存空间

4.4.3 实现示例

C 语言缓存管理:

```
1| // 从缓存复制数据到用户buffer
2| if (s_up_data.recv_data_size > 0) {
3|     uint32_t copy_len = min(s_up_data.recv_data_size, size);
4|     memcpy(buf, &s_up_data.recv_buf[s_up_data.recv_data_off], copy_len);
5|     size -= copy_len;
6|     buf += copy_len;
7|     s_up_data.recv_data_size -= copy_len;
8|     s_up_data.recv_data_off += copy_len;
9| }
```

Python 缓存管理:

```
1| # 从缓存读取数据
2| if self.__recv_size > 0:
3|     copy_len = self.__recv_size if self.__recv_size < size else size
4|     ret.append(bytes(self.__recv_buf[self.__recv_off : self.__recv_off + copy_len]))
5|     size -= copy_len
6|     self.__recv_size -= copy_len
7|     self.__recv_off += copy_len
```

4.4.4 模式切换管理

协议支持发送/接收模式的动态切换:

1. **模式状态跟踪**: 使用状态标志跟踪当前的工作模式
2. **自动模式切换**:
 - 发送数据时, 如果当前处于接收模式, 先发送 SWITCH 帧切换
 - 接收数据时, 如果当前处于发送模式, 先发送 SWITCH 帧切换
3. **模式同步**: 通过 SWITCH 帧与对端协商模式切换, 确保双方状态一致

5 通信流程

5.1 初始化与同步流程

协议采用主从式初始化模式, 确保通信双方建立可靠连接:

5.1.1 初始化流程

表 11 初始化流程

步骤	主机 (send_mode=true)	从机 (send_mode=false)
1	清空 UART 接收缓存	清空 UART 接收缓存
2	发送 RESYNC_REQ 帧	等待 RESYNC_REQ 帧
3	等待 RESYNC_ACK 响应	收到 RESYNC_REQ 后发送 RESYNC_ACK
4	收到 RESYNC_ACK, 初始化完成	发送 RESYNC_ACK 后, 初始化完成
5	进入发送模式	进入接收模式

5.1.2 重新同步机制

当通信过程中发生严重错误时, 任一方可发起重新同步:

1. 触发条件:

- 连续重传失败超过最大次数
- 检测到协议状态不一致
- 长时间无响应

2. 同步流程:

- 发送 RESYNC_REQ 帧 (包含期望的模式位)
- 等待对方 RESYNC_ACK 响应
- 重置协议状态为初始状态
- 根据协商结果设置工作模式

3. 同步优化:

- RESYNC_REQ 帧的 bit0 位指示发送方期望的模式
- 接收方响应 RESYNC_ACK 后采用对应的模式
- 避免模式冲突和死锁

5.1.3 首次通信优化

协议实现了首次通信的优化机制:

```
1| // C语言实现
2| if (s_up_data.first_init) {
3|     s_up_data.first_init = FALSE;
4|     if (!is_send_mode() && type == TYPE_ACK) {
5|         // 接收方首次ACK不发送, 减少不必要的通信
6|         ret = STP_OK;
7|     } else {
8|         ret = up_send_frame(type, data, size);
9|     }
10| }
```

```
1| # Python实现
2| if self.__init:
3|     self.__init = False
4|     if not self.__send_mode and type == TYPE_ACK:
5|         pass # 首次ACK跳过发送
6|     else:
7|         self.__send_frame(type, data)
```

这种优化避免了接收方在初始状态下发送无意义的 ACK 帧。

5.2 数据传输流程

5.2.1 发送端流程

1. **模式检查**: 检查当前是否为发送模式, 如果不是则先切换模式
2. **数据分块**: 将大数据按最大负载大小 (255 字节) 分块
3. **逐块发送**:
 - 发送 DATA 帧
 - 等待 ACK 确认
 - 处理可能的错误重传
 - 继续下一块数据

5.2.2 接收端流程

1. **模式切换**: 如果当前为发送模式, 先发送 SWITCH 帧切换
2. **数据接收**:
 - 发送 ACK 请求数据
 - 接收 DATA 帧
 - 校验数据完整性
 - 缓存或直接返回给用户
3. **数据拼接**: 自动拼接多个数据帧, 对用户透明

6 总结

本文档详细描述了 UART STP 协议的设计理念和实现架构。该协议具有以下特点:

1. **可靠性**: 基于 CRC16 校验和自动重传机制确保数据传输完整性
2. **灵活性**: 支持任意长度数据传输, 通过自动分块机制处理大数据
3. **高效性**: 优化的帧结构设计, 最大负载 255 字节, 协议开销低
4. **状态机驱动**: 清晰的半双工通信状态管理和模式切换机制
5. **分层架构**: 通过 HAL 抽象层实现硬件无关的协议设计

该协议设计充分考虑了串口通信的特点和限制，通过合理的帧结构、状态机设计和错误处理机制，为上层应用提供了可靠的数据传输保证。协议实现了 C 语言和 Python 两个版本，支持跨语言和跨平台的应用场景。